

Comandos slash y selección de modelo

Claude Certified Developer, Foundation (CCDV-F) · Masterclass UST

ES Sesión 5 de 12 · Notas en español (es-ES)

El mismo repaso que cubrió los modos de permiso cubrió los **comandos slash**: los integrados de Claude Code (`/model`, `compact` y el resto de la lista del producto) frente a las **skills personalizadas** que aparecen como entradas slash cuando añadís un `SKILL.md`. La selección de modelo no es otro producto: es un slash (o un control «switch model») que ata la sesión actual, y más adelante cada subagente, a un modelo de clase Haiku / Sonnet / Opus que vuestra org puede usar de verdad.

LA PALETA DE COMANDOS SE MUEVE

La UI en directo mostró un valor por defecto en torno a «Sonnet 5» y un selector `/model` de lo que tuviera la org. Los nombres integrados (`/compact`, `/clear`, `agents`, `mcp`, `hooks`) cambian entre versiones. Leed el menú `/` de la sesión y la **documentación de Claude Code** en vez de memorizar una captura del 12 de agosto.

/model — listar o fijar, no adivinar SKUs

Escribís `/model` (el formador también dijo `/models`) para listar los modelos del espacio de trabajo, o `/model` más un nombre para cambiar. A menudo hay un control switch-model con el valor actual. Puntos didácticos:

- La disponibilidad es organizativa: no podéis elegir un modelo al que el workspace no tenga derecho.
- Usad un modelo más fuerte (clase Opus en la clase) para planificación / subagentes de arquitectura; clase Sonnet para generación rutinaria; clase Haiku cuando la tarea es clasificación barata o boilerplate — comprobad siempre la escalera vigente.
- Los subagentes pueden *heredar* el modelo del padre o fijar el suyo en el markdown del agente (M05-C03). Un agente de Dockerfile no necesita el mismo modelo que una skill de arquitecto principal.
- Cambiar de modelo no vacía `CLAUDE.md`; sí cambia coste y latencia de los turnos siguientes.

Slashes integrados frente a slashes de skill

Tipo	Ejemplos de clase	Quién lo define	Cuándo corre
Integrado en Claude Code	<code>/model</code> , <code>compact</code> (y otras utilidades de sesión que liste el menú <code>/</code>)	El producto	Cuando los escribís; configuran el cliente.
Slash de skill	<code>/business-analyst</code> , <code>/architect</code> , <code>/backend</code> , <code>/frontend</code> , skills tipo Docker más adelante	Carpetas de skill del repo (<code>SKILL.md</code>)	Con el slash explícito, o en automático si no se desactivó la invocación por el modelo.
Mención / delegación de agente	Review the code → agente code-reviewer; generate Dockerfile → agente docker	<code>`.claude/agents`</code> (o ámbito de usuario/plugin)	El orquestador encaja la descripción, o invocáis el agente.

Las skills son «comandos slash personalizados» en el vocabulario del formador. No son MCP. Inyectan un paquete de rol para un turno (o mientras las instrucciones de la skill estén en contexto). Compact es el tipo contrario de comando: reduce el contexto de sesión para seguir sin partir de un chat en blanco — ligado a la ventana de contexto del módulo 03, no a elegir Opus.

Las superficies de configuración sobre las que se sienta el menú /

Los slash son cómo *conducís* Claude Code; la configuración sigue en ficheros: `CLAUDE .md` de proyecto, `CLAUDE.md` anidados, `settings .json` (hooks en el módulo 06), árboles de skills, markdown de agentes. El formador apuntó también a la configuración general de Claude Code: memoria por ruta, modo de permiso, modelo. Tratad la barra slash como la API interactiva de esos ficheros, no como una segunda fuente de verdad.

Pedirle a Claude que escriba el slash

Más tarde en la misma sesión el formador pidió a Claude *crear una skill* (p. ej. Docker/contenedores) en vez de escribir a mano cada `SKILL.md`. Está permitido: la skill generada aparece como slash igual que las hechas a mano. Acotad el prompt («solo el contenedor frontend, no arranques Docker Desktop ni os comáis el mundo») o la skill nueva heredaré el mismo bucle demasiado entusiasta que queríais empaquetar. Desactivad la invocación por el modelo si necesitáis que no dispare nunca salvo con slash.

El contexto del prompt no es solo el slash

Cuando hacéis slash de frontend y pegáis una captura de filtros, el modelo sigue cargando el `CLAUDE.md` de frontend. Los mejores prompts nombran la ruta o el componente (`/tasks`) para no gastar tokens buscando por toda la SPA. Los comandos de arranque del README pueden ser una instrucción en el momento del slash: «consulta el README y arranca la app» en vez de volcar cada script npm en la skill.

FRASE LISTA PARA EL EXAMEN

`/model` elige el cerebro; los slash de skill eligen el rol; el modo de permiso elige cada cuánto el cerebro debe pedir permiso antes de usar tools. Mezclar esas tres cosas es un fallo habitual en las respuestas escritas.

Qué no meter en un slash

No peguéis datos de cliente. No confiéis en un slash para imponer Prettier o el formato de empresa: eso es un hook local (módulo 06) para que no facture tokens. No creéis una skill que en realidad sea un servidor MCP; si necesitáis Figma o una API interna, son tools/MCP, opcionalmente *permitidas* en un agente, no un `SKILL.md` que finja iniciar sesión.

Documentación oficial

• Visión general de Claude Code

<https://code.claude.com/docs/en/overview>

Menú slash, skills y comandos de sesión, incluido el modelo.

- **Modelos**

<https://docs.claude.com/en/docs/about-claude/models>

Qué IDs puede listar /model esta semana.

- **Workspaces**

<https://docs.claude.com/en/docs/build-with-claude/workspaces>

Por qué dos usuarios UST pueden ver listas de modelos distintas.

- **Visión general de prompting**

<https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview>

Las skills son prompts empaquetados; compact es higiene de contexto.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.